

Pintos Project 4: Virtual Memory

담당 교수 : 김영재

이름 / 학번 : 방지혁 / 20190808

개발 기간 : 2025/11/24 ~ 2025/12/21

I. 개발 목표

우리가 기존까지 했던 로딩은 굉장히 비효율적이었다. 수업에서 배웠듯 프로그램 전체를 그대로 physical memory에 올리기에 여러 프로그램을 올리는데 한계가 존재했다. 이를 수정하는 것이 주 목표이다. 이들을 나누어 lazy loading 하여 정말 필요한 것만 순차적으로 올려지도록 구현한다. 이 과정에서 page table 구현과 page fault handler를 수정한다. 그리고, physical memory에도 한계가 있기 때문에 LRU 알고리즘을 통해 swap이 이뤄질 수 있도록 해야 한다. 또한, stack이 자랄 수 있게 구현해야 한다.

II. 개발 범위 및 내용

1. 개발 범위

1. Page Table & Page Fault Handler

프로세스의 가상 주소를 관리하고 물리메모리와의 매핑 정보를 유지하기 위해 page table이 필요하다. 이를 통해 한꺼번에 메모리에 올리지 않고, 필요한 것만 로드하는 demand paging이 가능해진다. 다만 필요 데이터가 메모리가 존재하지 않는 page fault가 발생하는 경우 이를 디스크에서 가져올 수 있다.

2. Disk Swap

물리 메모리가 가득 찼을 때 사용하지 않는 페이지는 다시 디스크로 내보내 이를 통해 우리가 필요로 하는 페이지를 위한 공간을 확보할 수 있다. 이를 통해 실제 physical memory보다 더 큰 주소 공간을 필요로 하는 프로그램들을 동시에 실행할 수 있게 된다.

3. Stack Growth

초기에 할당된 스택의 크기를 초과할 때 스택을 확장할 수 있게 된다. Page fault가 일어난 원인이 stack 부분 접근을 한 경우이다. 이러한 경우 무작정 확장하는 것은 아니고 실제 물리 크기에 대한 확인을 거치고 확장하여 스택 오버플로우가 나지 않게 된다.

2. 개발 내용

1. Page fault가 발생하는 이유와 이를 handling하는 전반적인 과정을 서술

프로세스가 접근하려는 가상주소가 현재 물리 메모리에 올라와 있지 않거나, 유효하지 않은 주소에 접근하려고 할 경우 발생한다. 과정에 대해 서술하자면 예외가 발생하면 page_fault() 핸들러에게 제어권이 넘어간다. 이 핸들러는 해당 주소에 해당하

는 vm_entry를 검색한다. 그리고 유효한 주소라면 alloc_page 함수를 통해 물리 메모리를 할당 받는다. 그리고 할당 받은 공간에 데이터를 읽어온다. 가상주소와 물리 주소를 매핑하고, fault가 일어나기 전 명령어로 돌아가 재실행한다.

2. Disk swap 발생 시 사용한 page replacement algorithm에 대해 서술

Second chance algorithm을 활용하였다. 모든 프레임들은 전역 변수인 원형 리스트 lru_list를 통해 관리된다. 각 페이지마다 bit를 확인하여 1이면 0으로 바꾸고 한 번 더 기회를 주어 다음페이지로 넘어간다. 0이라면 해당 페이지를 victim으로 결정하여 swap-out 한다.

3. Stack growth 구현 시 stack 확장 여부를 판단할 수 있는 방법에 대해 서술

Esp 기준으로 접근 주소가 esp - 32보다 크거나 같은지 확인한다. 해당 주소가 유효한 유저 영역 주소 공간 내에 있고 최대 스택 크기를 초과하지 않는지 검사하여 만약 해당 조건을 부합한다면 expand_stack()을 통해 스택 영역을 확장한다.

III. 추진 일정 및 개발 방법

1. 추진 일정

전체 일정: 2025/11/24 ~ 2025/12/21

- 2025/11/24 ~ 2025/11/30: page fault 관련 부분 구현
- 2025/12/1 ~ 2025/12/7: disk swap 관련 부분 구현
- 2025/12/8 ~ 2025/12/14: stack growth 관련 부분 구현
- 2025/12/18 ~ 2025/12/21: 보고서 작성

2. 개발 방법

1. Page Table & Page Fault Handler

userprog/exception.c, userprog/process.c, vm/page.c, vm/frame.c를 수정해야 한다. 각 프로세스별 가상 주소를 관리할 SPT(Supplementary Page Table)과 물리 메모리 정보를 담은 frame table(struct frame_entry)가 필요하다. 또한, Page_fault시 exception.c의 page_fault()함수를 수정하여 handle_mm_fault를 호출하도록 구현해야 하며, supplementary page table을 위한 insert_vm_entry, find_vm_entry 같은 삽입 삭제 연산등이 필요하다. 또한, 이 vm_entry는 hash 자료구조를 이용한다. 해당 page

table은 start_process에서 초기화 될 수 있도록 해야한다. 그리고 thread 구조체에 멤버 변수로 존재해야한다. 그리고 load_segment 함수에서 page table entry가 생성되어야 한다.

2. Disk Swap

Vm 디렉토리 안에 swap.c, frame.c page.c가 필요하다. 스왑 영역을 비트 단위로 관리하는 bitmap 자료구조가 필요하다. 해당 스왑은 우리가 수업에서 배웠던 second chance algorithm에 기반하여 동작한다. Frame table이 필요하며 vm_entry에는 swap_slot 필드가 추가되어야 한다. 그리고 물리 메모리가 부족할 때 페이지를 내보내는 try_to_free_pages(), swap_out(), swap_in() 같은 함수도 구현해야한다.

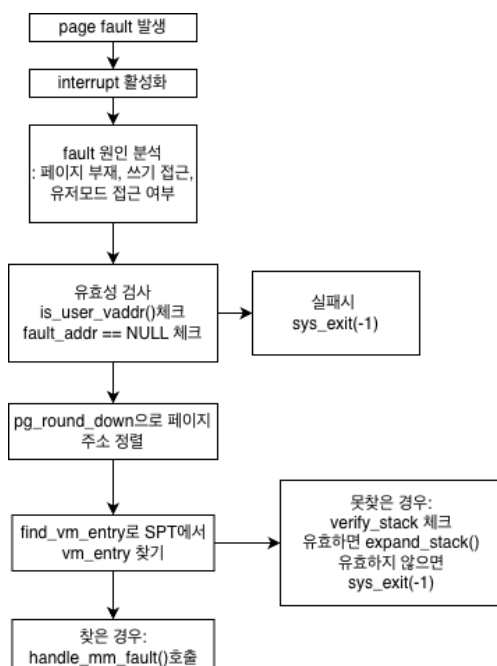
3. Stack Growth

Exception.c, page.c, syscall.c에 대한 수정이 필요하다. Vm_entry를 생성할 때 해당 페이지가 스택 용도임을 구분하기 위해서 VM_ANON 타입이 필요하다. 접근 주소에 대해 스택 확장 대상인지 확인하는 verify_stack()과 실제로 페이지를 동적으로 할당하는 expand_stack() 함수가 필요하다. 그리고 시스템 콜에서 전달된 버퍼 주소가 스택 영역일 경우 check_address()에서 이를 확인할 수 있어야 한다.

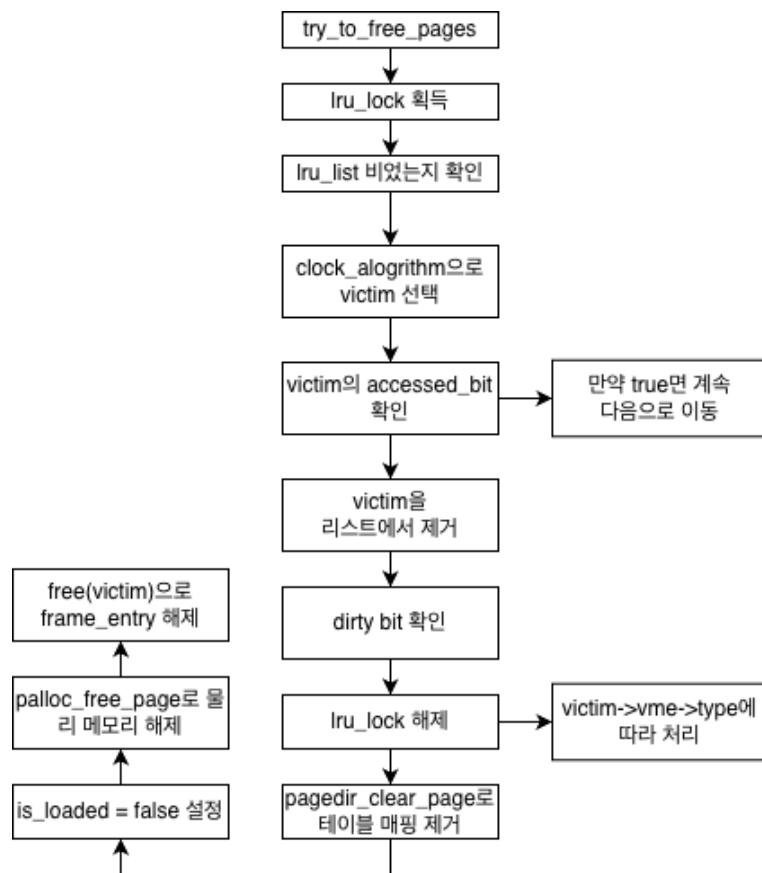
IV. 연구 결과

1. Flow Chart

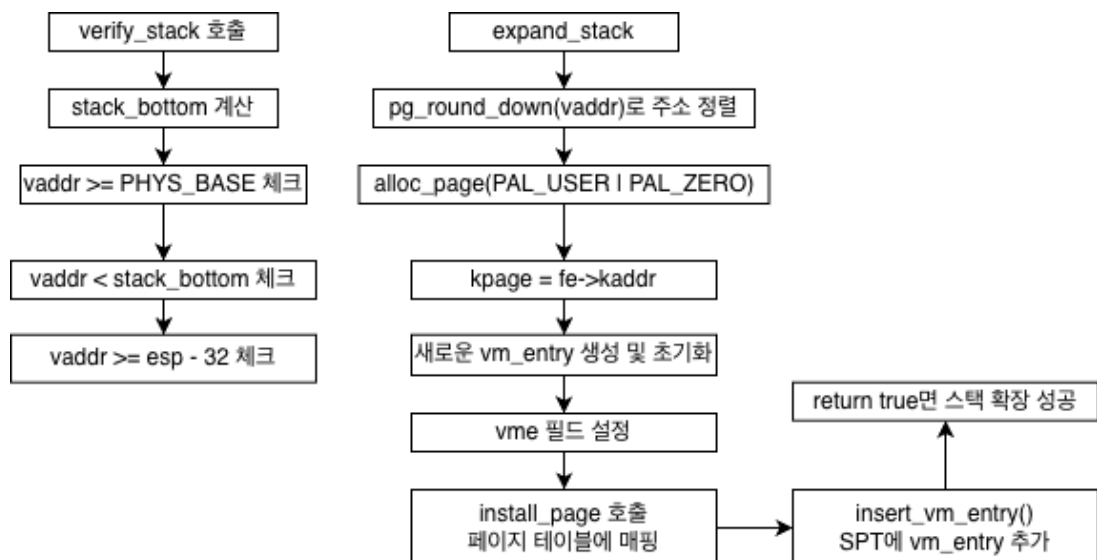
- Page Table & Page Fault Handler



- Disk Swap



- Stack Growth



2. 제작 내용

1. Page Table & Page Fault Handler

우선 SPT가 제일 중요하다. 해시테이블 기반으로 vm_entry는 메타데이터를 저장한다.

```
struct vm_entry {
    uint8_t type; // VM_BINARY, VM_FILE, VM_ANON

    size_t offset; // 읽어야 할 파일 offset
    size_t read_bytes; // 가상페이지에 쓰여져 있는 데이터 크기
    size_t zero_bytes; // 0으로 채울 남은 페이지의 byte

    bool writable; // True이면 write 가능, False이면 write 불가능
    bool is_loaded; // 물리 메모리 탑재 여부

    void* vaddr; // vm_entry의 가상페이지 번호
    struct file* mapped_file; // 가상주소와 매핑된 파일

    // struct list_elem mmap_elem;
    size_t swap_slot; // swap slot

    struct hash_elem hash_elem; // hash table element
};
```

구조체 멤버 변수에 대한 설명은 주석에 있으므로 생략하겠다.

```
static unsigned vm_entry_hash_func(const struct hash_elem* e, void* aux UNUSED) {
    // hash_entry()로 vm_entry 구조체 검색
    // hash_int()로 vaddr에 대한 해시값 구하기
    struct vm_entry* target = hash_entry(e, struct vm_entry, hash_elem);
    return hash_int((int)target->vaddr);
}
```

Hash_int()를 사용하여 가상 주소를 해시 값으로 변환하는 함수이다.

```
// vm_entry_less_func
static bool vm_entry_less_func(const struct hash_elem* a,
                               const struct hash_elem* b, void* aux UNUSED) {
    // hash_entry()로 vm_entry 구조체 얻고 vaddr 비교
    struct vm_entry* target_a = hash_entry(a, struct vm_entry, hash_elem);
    struct vm_entry* target_b = hash_entry(b, struct vm_entry, hash_elem);

    return target_a->vaddr < target_b->vaddr;
}
```

가상 주소를 기준으로 vm_entry를 정렬한다.

```
// vm_init
void vm_init(struct hash* vm) {
    // hash_init으로 해시 테이블 초기화
    // 인자로 vm_hash_func & vm_less_func
    hash_init(vm, vm_entry_hash_func, vm_entry_less_func, NULL);
}
```

Vm_init에서는 hash_init을 호출하여 프로세스 생성시 SPT를 초기화한다.

```

static bool load_segment(struct file* file, off_t ofs, uint8_t* upage,
                        uint32_t read_bytes, uint32_t zero_bytes,
                        bool writable) {
    ASSERT((read_bytes + zero_bytes) % PGSIZE == 0);
    ASSERT(pg_ofs(upage) == 0);
    ASSERT(ofs % PGSIZE == 0);

    // file_seek(file, ofs);
    while (read_bytes > 0 || zero_bytes > 0) {
        /* Calculate how to fill this page.
         * We will read PAGE_READ_BYTES bytes from FILE
         * and zero the final PAGE_ZERO_BYTES bytes. */
        size_t page_read_bytes = read_bytes < PGSIZE ? read_bytes : PGSIZE;
        size_t page_zero_bytes = PGSIZE - page_read_bytes;

        /* Get a page of memory. */
        struct vm_entry* vme = malloc(sizeof(struct vm_entry));
        if (vme == NULL) return false;

        vme->mapped_file = file_reopen(file);
        if (vme->mapped_file == NULL) {
            free(vme);
            return false;
        }

        vme->type = VM_BINARY;
        vme->vaddr = upage;
        vme->writable = writable;
        vme->is_loaded = false;
        vme->offset = ofs;
        vme->read_bytes = page_read_bytes;
        vme->zero_bytes = page_zero_bytes;
        vme->swap_slot = 0;

        /* Add the page to the process's address space. */
        if (!insert_vm_entry(&thread_current()->vm, vme)) {
            file_close(vme->mapped_file);
            free(vme);
            return false;
        }
        /* Advance. */
        read_bytes -= page_read_bytes;
        zero_bytes -= page_zero_bytes;
    }
}

```

또한 기존의 코드를 변경해야하는데 load_segment 과정에서 실제 물리 프레임을 할당하지 않고 vm_entry를 SPT에 등록만 하는 lazy loading을 수행하도록 바꾼다.

```

if (!is_user_vaddr(fault_addr) || fault_addr == NULL) {
    sys_exit(-1);
    return;
}

if (!not_present) {
    sys_exit(-1);
    return;
}

```

```

struct vm_entry *vme = find_vm_entry(pg_round_down(fault_addr));
if (vme != NULL) {
    if (handle_mm_fault(vme)) return;
    else {
        sys_exit(-1);
        return;
    }
}

```

해당 코드들은 page_fault handler 코드의 일부분이다. 우선 커널 영역이거나 NULL 접근 시 종료하고, 페이지가 존재하는데 쓰기 보호를 위반한 경우와 같이 폴트 발생 시 종료한다. page fault 발생시 find_vm_entry를 통해 해당 주소의 정보를 찾고 Handle_mm_fault를 호출하여 물리 프레임을 할당하고 데이터를 적재한다.

```

bool handle_mm_fault(struct vm_entry* vme) {
    // 페이지 폴트 발생시 물리페이지를 할당 Disk에 있는 file을 물리페이지로 load
    // load_file (void* kaddr, struct vm_entry *vme)를 사용
    // 물리메모리에 적재가 완료되면 가상주소와 물리주소를 페이지테이블로 맵핑

    ASSERT(vme != NULL);
    if (vme->is_loaded) return true;
    struct frame_entry* fe = alloc_page(PAL_USER | PAL_ZERO);
    if (fe == NULL) return false;

    void* kpage = fe->kaddr;
    fe->vme = vme;

    uint8_t type = vme->type;

    if (type == VM_BINARY) {
        if (!load_file(kpage, vme)) {
            free_page(kpage);
            return false;
        }
    }
    } else if (type == VM_FILE) {
        if (!load_file(kpage, vme)) {
            free_page(kpage);
            return false;
        }
    }
    } else if (type == VM_ANON) {
        // swap에서 읽어오기
        swap_in(vme->swap_slot, kpage);
    }

    if (!install_page(vme->vaddr, kpage, vme->writable)) {
        free_page(kpage);
        return false;
    }
    vme->is_loaded = true;
    return true;
}

```

가장 중요한 것은 handle_mm_fault이다. 우선 이미 로드된 경우를 확인하여 중복 로드를 방지한다. 두 쓰레드가 동시에 같은 페이지를 접근하면 둘 다 페이지 폴트가 발생할 수 있기 때문이다. 물리 프레임을 할당해서 Frame과 vm_entry를 연결한다.

Alloc_page는 palloc_get_page를 호출하는 함수이다. 그리고 페이지 타입별로 데이터를 로드하는데 VM_BINARY는 실행 파일의 코드와 데이터이다. Read only 코드로 eviction할 경우 clean 이면 원본을 유지하고, dirty면 스왑으로 이동한다. VM_FILE은 mmap으로 생성된 매핑으로 구현하지는 않았다. 파일의 특정부분을 메모리에 매핑한 것으로 clean이면 그냥 버린다. 파일에 그대로 있기 때문이다. VM_ANON의 경우 스택, 힙 같은 경우이다. 이전에 스왑아웃된 페이지를 다시 불러온 경우이다. 그렇기에 swap_in을 사용한다. 그리고 install_page를 통해 모두 페이지 테이블 매핑을 해준다. 여기서 load_file은 참고로 디스크의 파일을 물리 메모리로 읽어오는 함수다.

2. Disk Swap

```
void swap_init(void) {
    swap_block = block_get_role(BLOCK_SWAP);
    if (swap_block == NULL) {
        return;
    }

    size_t swap_size = block_size(swap_block) / SECTORS_PER_PAGE;
    swap_bitmap = bitmap_create(swap_size);
    if (swap_bitmap == NULL) {
        PANIC("Swap bitmap creation failed");
        return;
    }
    bitmap_set_all(swap_bitmap, false);
}
```

시스템 부팅을 할 때 스왑 블록을 초기화하고 비트맵으로 슬롯 사용 여부를 관리한다. 해당 코드는 스왑 크기를 계산하고 비트맵을 생성하여 초기화하는 코드이다.

```
size_t swap_out(void *kaddr) {
    if (swap_block == NULL) {
        PANIC("Swap block device not available");
    }
    size_t slot = bitmap_scan_and_flip(swap_bitmap, 0, 1, false);
    if (slot == BITMAP_ERROR) {
        PANIC("SWAP FULL");
    }
    for (size_t i = 0; i < SECTORS_PER_PAGE; i++) {
        block_write(swap_block, slot * SECTORS_PER_PAGE + i,
                   kaddr + i * BLOCK_SECTOR_SIZE);
    }
    return slot;
}
```

Swap out은 물리 메모리의 페이지를 스왑 디스크로 내보내는 함수이다. 비트맵에서 빈 슬롯을 찾아 0에서 1로 변경시키고 빈 슬롯을 찾아 8개의 섹터에 나눠 쓴다. 그리고 슬롯 번호를 반환한다. 이는 나중에 swap_in 에 사용된다.

```
void swap_in(size_t slot, void *kaddr) {
    if (swap_block == NULL || !bitmap_test(swap_bitmap, slot)) {
        PANIC("Invalid swap slot");
    }
    for (int i = 0; i < SECTORS_PER_PAGE; i++) {
        block_read(swap_block, slot * SECTORS_PER_PAGE + i,
                  kaddr + i * BLOCK_SECTOR_SIZE);
    }
    bitmap_set(swap_bitmap, slot, false);
}
```

이와 반대로 swap in은 유효성 검사를 하고 8개의 섹터를 순차적으로 읽는다. 그리고 슬롯을 다시 사용 가능하도록 표시하는 함수로 페이지를 읽어 물리 메모리로 복원하는 것이다.

가장 중요한 것은 frame eviction이다. 물리 메모리가 부족할 때 victim 페이지를 선택하여 앞서 서술했던 swap out을 하는 것이다.

```

/* eviction */
void try_to_free_pages(enum pallof_flags flags) {
    lock_acquire(&lru_lock);
    while (true) {
        if (list_empty(&lru_list)) {
            lock_release(&lru_lock);
            return;
        }

        if (lru_clock == NULL || lru_clock == list_end(&lru_list)) {
            lru_clock = list_begin(&lru_list);
        }

        struct list_elem *e = lru_clock;
        lru_clock = list_next(lru_clock);

        struct frame_entry *victim = list_entry(e, struct frame_entry, lru_elem);

        if (pagedir_is_accessed(victim->t->pagedir, victim->vme->vaddr)) {
            pagedir_set_accessed(victim->t->pagedir, victim->vme->vaddr, false);
            continue;
        }

        list_remove(&victim->lru_elem);

        bool dirty = pagedir_is_dirty(victim->t->pagedir, victim->vme->vaddr);

        lock_release(&lru_lock);

        switch(victim->vme->type) {
            case VM_ANON:
                victim->vme->swap_slot = swap_out(victim->kaddr);
                break;
            case VM_FILE:
                if (dirty) {
                    lock_acquire(&filesys_lock);
                    file_write_at(victim->vme->mapped_file, victim->kaddr, victim->vme->read_bytes, victim->vme->offset);
                    lock_release(&filesys_lock);
                }
                break;
            case VM_BINARY:
                if (dirty) {
                    victim->vme->swap_slot = swap_out(victim->kaddr);
                    victim->vme->type = VM_ANON;
                }
                break;
        }

        pagedir_clear_page(victim->t->pagedir, victim->vme->vaddr);
        victim->vme->is_loaded = false;
        pallof_free_page(victim->kaddr);
        free(victim);

        // lock_acquire(&lru_lock);
        return;
    }
}

```

Lru_list를 순회하여 최근에 사용되지 않은 페이지를 찾는다. Lru_clock이라는 포인터를 사용하여 element를 하나씩 검사한다. 하드웨어가 관리하는 accessed bit 값을 pagedir_is_accessed함수로 확인한다. 만약 1이라면 0으로 초기화하고 다음 페이지로 넘어가지만 0이라면 이를 victim으로 결정한다. 또한 페이지 타입에 따라 처리 방식이 다르다. VM_ANON의 경우 swap out을 한다. 스택이나 힙은 파일 백업이 없는 페이지이기에 swap_out을 통해 빈 슬롯으로 데이터를 옮긴다. VM_FILE의 경우 write back한다. Dirty bit가 1이라는 것은 Mmap 된 파일 페이지가 중간에 수정되었다는 것이기에 file_write_at을 호출하여 원본 파일에 변경 내용을 추가한다. 그렇지 않을 경우 그냥 버린다. 그리고 VM_BINARY의 경우 swap out을 하고 타입을 바꾼다. 마찬가지로 실행 파일의 데이터가 수정되었다면 스왑영역으로 보내고 타입을 바꾸어 스왑에서 관리되도록 한다. 그리고 데이터 보관이 최종적으로 완료되면 pagedir_clear_page를 호출하여 가상과 물리 주소의 매핑을 끊고 vm_entry의 is_loaded flag를 false로 설정한다. 이를 통해 나중에 다시 접근할때 Page fault를 토 앵 다시 로드될 수 있다. 그리고 pallof_free_page를 통해 실제 물리 프레임을 커널에 반납하고 구조체를 free한다. 이를 위해 lru_lock이 중요하다. 이 때문에 데드락 상태에 봉착하여 어려움이 있었다.

3. Stack Growth

```
struct vm_entry *vme = find_vm_entry(pg_round_down(fault_addr));
if (vme != NULL) {
    if (handle_mm_fault(vme)) return;
    else {
        sys_exit(-1);
        return;
    }
}
else {
    if (verify_stack(fault_addr, f->esp)) {
        if (expand_stack(fault_addr)) return;
    }
    sys_exit(-1);
    return;
}
```

아까전에 서술했던 pagefault핸들러에서 사용자가 아직 할당되지 않은 스택 영역의 주소에 접근하면 page fault가 발생한다. 이에 핸들러는 유효한 요청인지 확인한다. 이는 verify_stack을 통해 이루어진다.

```
bool verify_stack(void* vaddr, void *esp) {
    void *stack_bottom = (void *)((uint8_t *)PHYS_BASE - (8 * 1024 * 1024));

    if (vaddr >= PHYS_BASE) return false;
    if (vaddr < stack_bottom) return false;

    if (vaddr >= (uint8_t *)esp - 32) return true;

    return false;
}
```

3가지 조건을 검사하는데 첫번째가 사용자 영역 여부이다. PHYS_BASE보다 아래에 있어야 한다는 것이다. 두번째는 최대 스택 크기에 제한이 있기에 PHYS_BASE - 8MB 보다 높은 주소이어야 한다. 마지막으로 push 명령어는 esp를 먼저 줄이는데 fault_addr가 현재 esp보다 낮을 수 있기에 esp - 32까지는 정상으로 간주한다.

그리고 유효성이 확인되면 확장한다.

```

bool expand_stack(void* vaddr) {

    void* page_addr = pg_round_down(vaddr);

    struct frame_entry *fe = alloc_page(PAL_USER | PAL_ZERO);
    if (fe == NULL) return false;
    void *kpage = fe->kaddr;

    // vme 생성 및 초기화
    struct vm_entry* vme = malloc(sizeof(struct vm_entry));
    if (vme == NULL) {
        free_page(kpage);
        return false;
    }
    vme->type = VM_ANON;
    vme->vaddr = page_addr;
    vme->writable = true;
    vme->is_loaded = true; // 스택은 즉시 로드됨
    vme->mapped_file = NULL;
    vme->offset = 0;
    vme->read_bytes = 0;
    vme->zero_bytes = 0;
    vme->swap_slot = 0;

    fe->vme = vme;

    if (!install_page(page_addr, kpage, true)) {
        free_page(kpage);
        free(vme);
        return false;
    }

    if (!insert_vm_entry(&thread_current()->vm, vme)) {
        free_page(kpage);
        pagedir_clear_page(thread_current()->pagedir, page_addr);
        free(vme);
        return false;
    }

    return true;
}

```

Lazy loading이 아니며 fault가 일어날 경우 Alloc_page를 통해 바로 물리 프레임이 할당된다. 새로운 vm_entry 생성 및 초기화가 일어나고 백업 파일이 없기에 VM_ANON이다. 페이지 테이블에 가상 주소와 물리 주소 매핑을 등록하고 SPT에 vm_entry를 삽입한다. 이를 통해 스택 확장이 일어나는 것이다.

4. 시험 및 평가 내용

```
pass tests/vm/pt-grow-stack
pass tests/vm/pt-grow-pusha
pass tests/vm/pt-grow-bad
pass tests/vm/pt-big-stk-obj
pass tests/vm/pt-bad-addr
pass tests/vm/pt-bad-read
pass tests/vm/pt-write-code
pass tests/vm/pt-write-code2
pass tests/vm/pt-grow-stk-sc
pass tests/vm/page-linear
pass tests/vm/page-parallel
pass tests/vm/page-merge-seq
pass tests/vm/page-merge-par
pass tests/vm/page-merge-stk
FAIL tests/vm/page-merge-mm
pass tests/vm/page-shuffle
```

Page-merge-mm 빼고는 다 통과하는 것을 확인할 수 있다.